

NAME	KEN KIBUGI MUIRURI
TRACKING NUMBER	CS-SA12-26014
COHORT	CYBERSHUJAA COHORT 1-2026
TASK	OWASP TOP 10(2025) REPORT

Contents

1.0	IAAA Failures.....	2
1.1	Broken Access Controls	2
1.2	Authentication Failures	3
1.3	Logging & Alerting Failures.....	4
2.0	Application Design Flaws	6
2.1	Security Misconfigurations	6
2.2	Software Supply Chain Failures	7
2.3	Cryptographic Failures.....	8
2.4	Insecure Design.....	9
3.0	Insecure Data Handling	10
3.1	Cryptographic Failures.....	10
3.2	Injection	11
3.3	Software or Data Integrity Failures	12
	Conclusion	14

OWASP TOP 10(2025) REPORT

Introduction

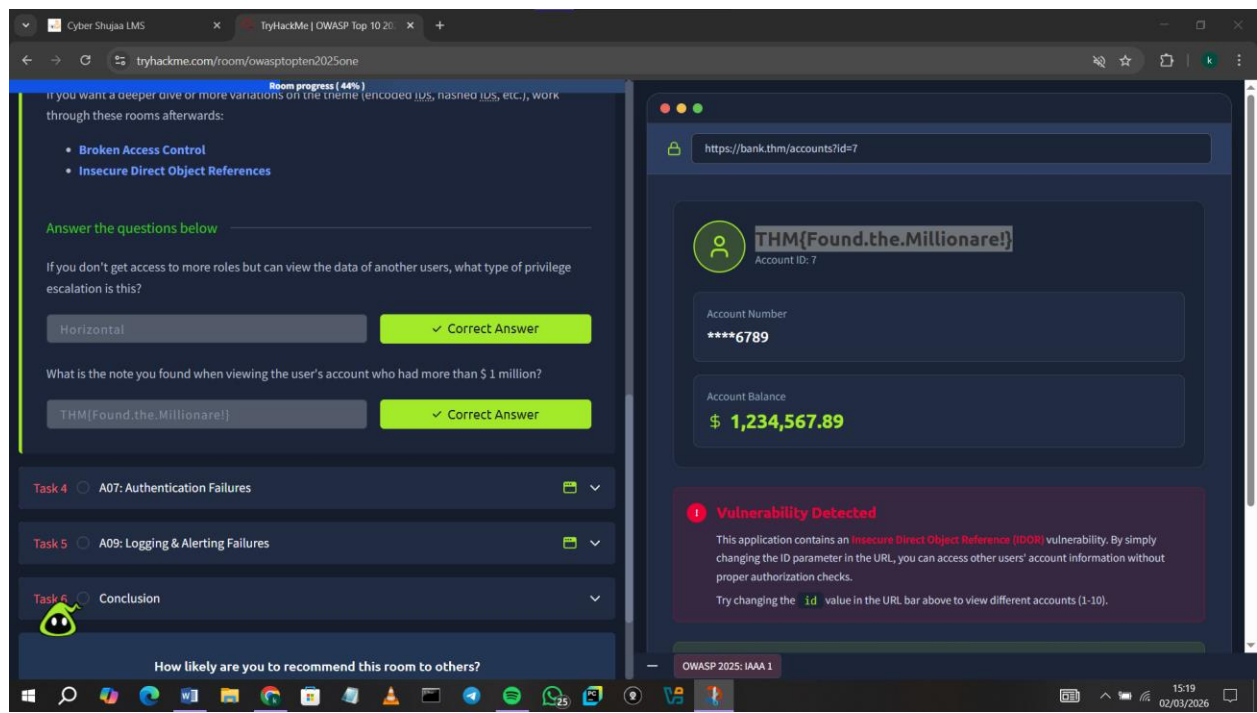
In this module, I explored the OWASP Top 10 (2025) and gained a deeper understanding of the most critical security risks affecting modern web applications. The module highlighted how vulnerabilities arise not only from coding errors, but also from insecure design, misconfigurations, weak authentication, poor cryptography, and failures in supply chains and integrity controls.

1.0 IAAA Failures

1.1 Broken Access Controls

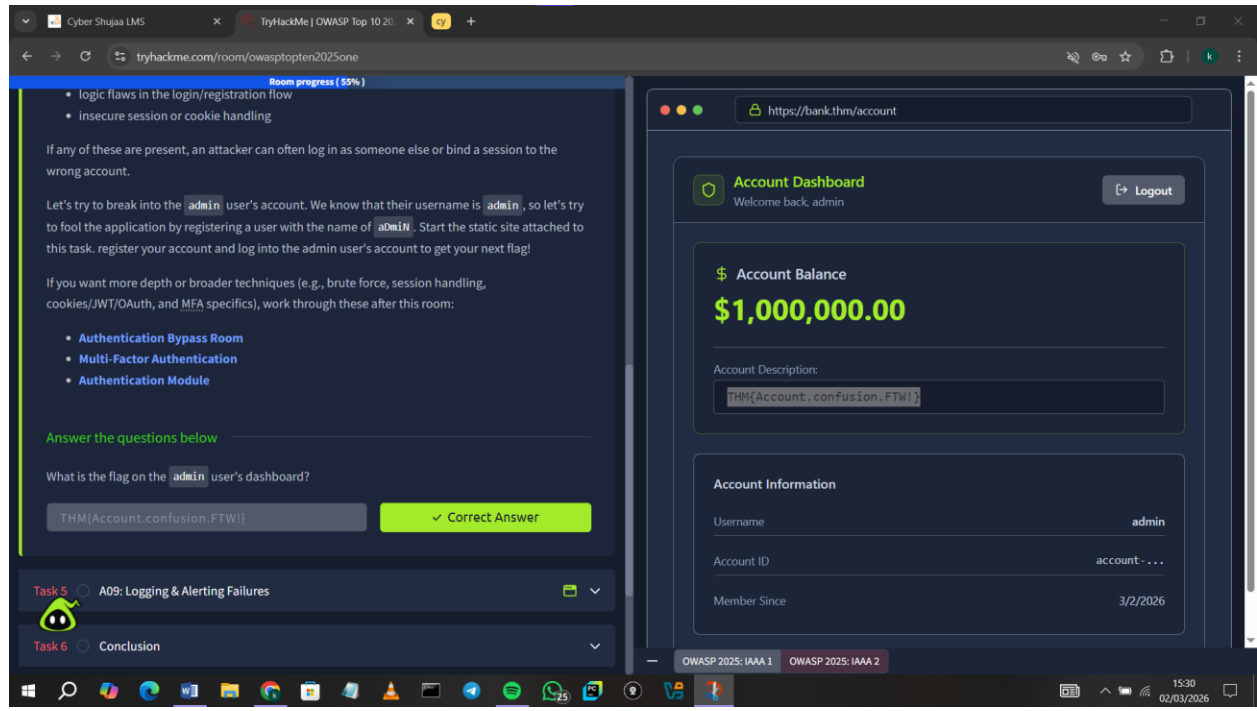
In this section, I learned that Broken Access Control occurs when applications fail to properly enforce user permissions on the server side. I understood how IDOR vulnerabilities allow attackers to manipulate URL parameters to access other users'

data. I also learned the difference between horizontal and vertical privilege escalation and why strict server-side authorization checks are essential.



1.2 Authentication Failures

In this section, I learned that Authentication Failures occur when applications improperly verify a user's identity. I understood how issues like username enumeration, weak passwords, logic flaws, and poor session handling can allow attackers to bypass login controls.

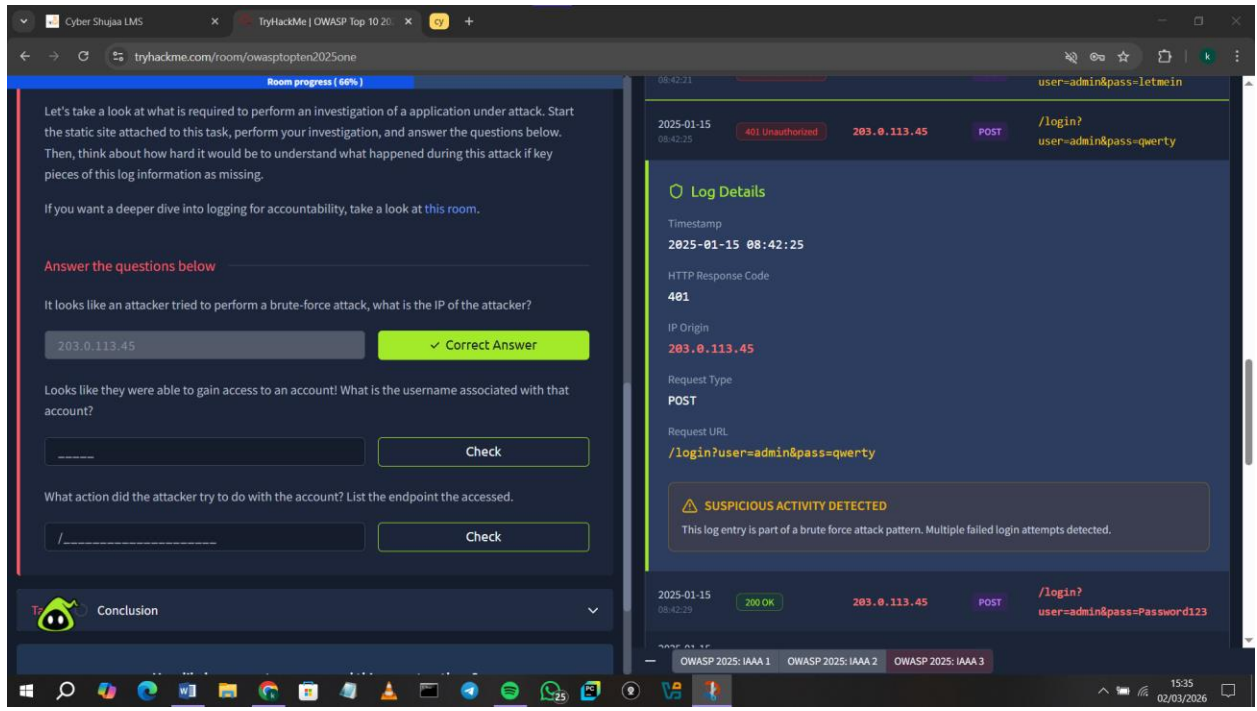


1.3 Logging & Alerting Failures

In this section, I learned that Logging and Monitoring Failures occur when applications do not properly record or alert on security-related events. I understood that effective logging ensures accountability by tracking who performed actions, when, and from where.

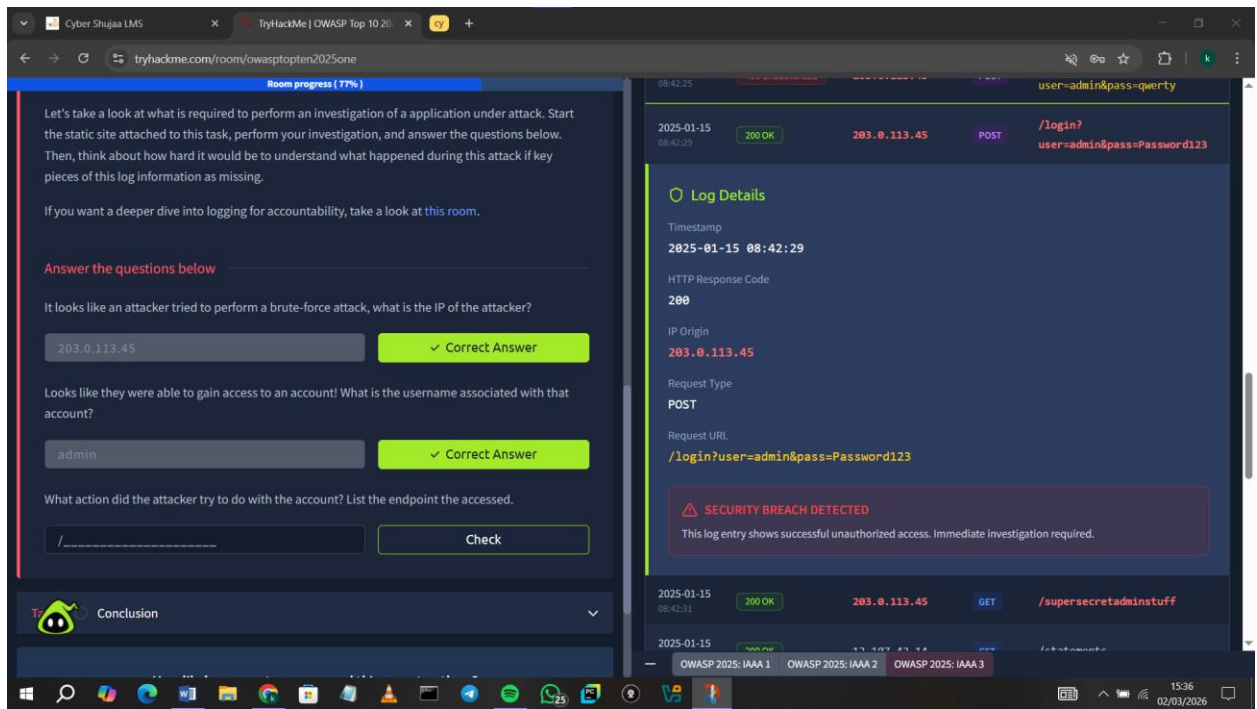
Question1: What is the IP of the attacker?

Answer: **203.0.113.45**



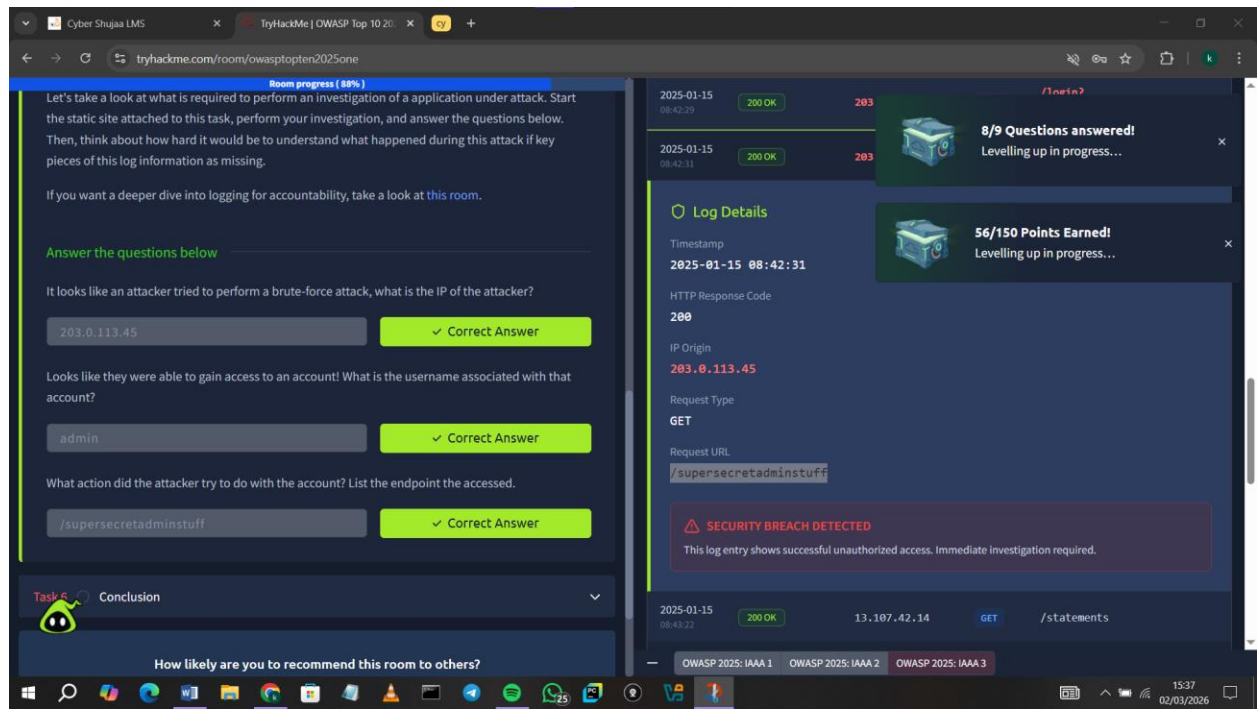
Question2: What is the username of the account they gained access to?

Answer: **admin**



Question3: List the endpoint accessed

Answer: **/supersecretadminstuff**



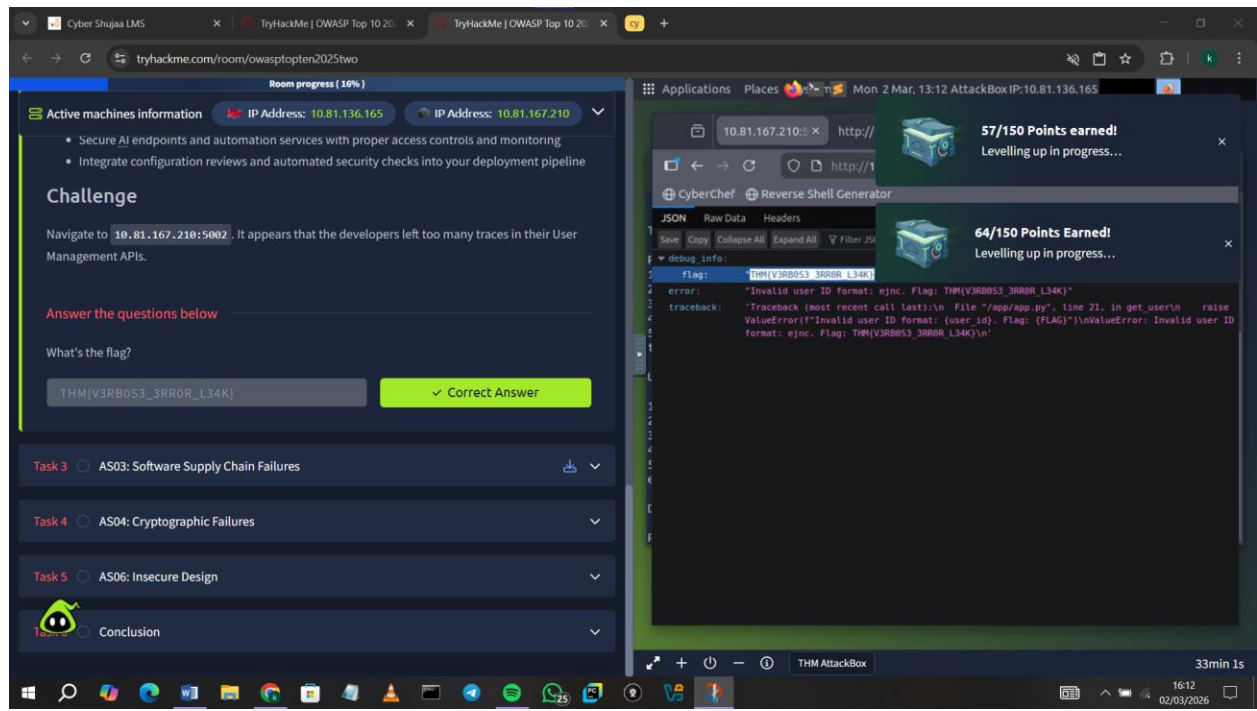
2.0 Application Design Flaws

2.1 Security Misconfigurations

In this section, I learned that Security Misconfigurations occur when systems or applications are deployed with unsafe default settings, exposed services, or improper permissions. I understood that even small configuration mistakes like public cloud storage or default credentials can lead to major data breaches. I also learned the importance of hardening configurations, limiting exposure, enforcing least privilege, and regularly auditing systems to prevent these risks.

Question: Navigate to MACHINE_IP:5002. It appears that the developers left too many traces in their User Management APIs. What's the flag?

Answer: To obtain the flag, I navigated to the address and used a non-numerical ID and the misconfiguration on the system helped me find it.

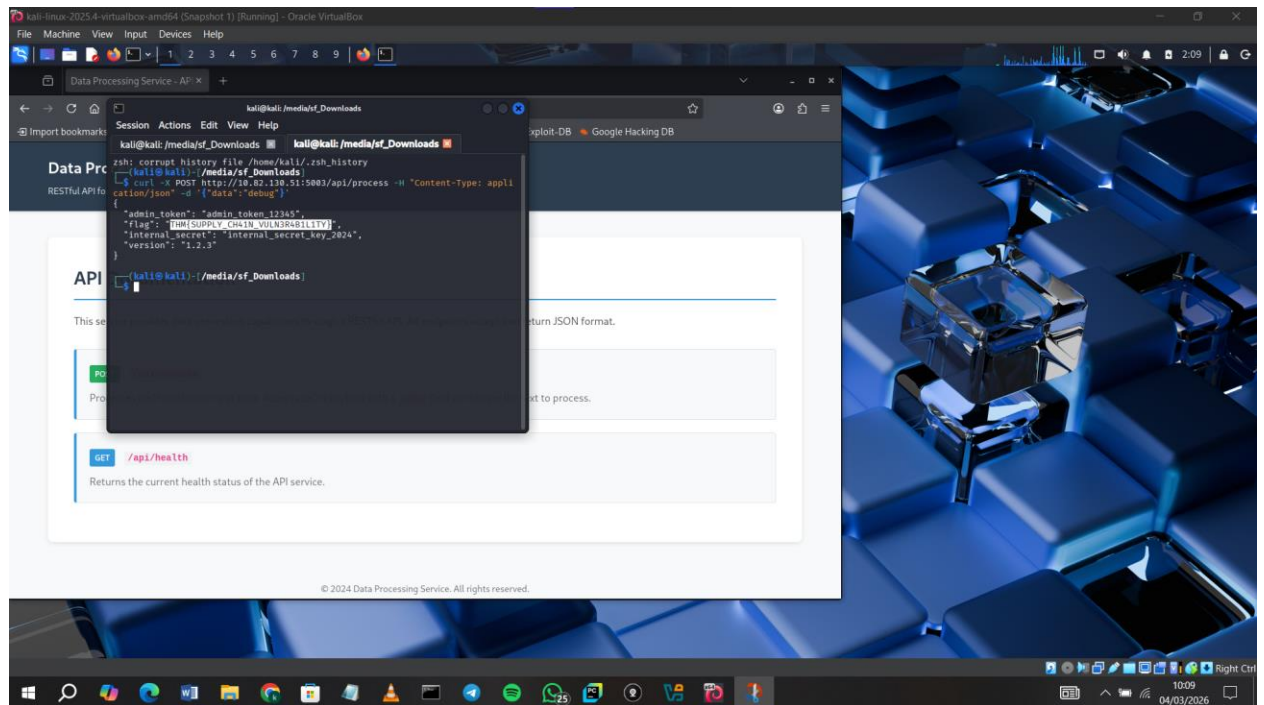


2.2 Software Supply Chain Failures

In this section, I learned that Software Supply Chain Failures occur when applications depend on compromised, outdated, or unverified third-party components. I understood that a single vulnerable dependency or tampered update can compromise an entire system, even if the core code is secure. I also learned the importance of verifying, monitoring, and securing dependencies and build pipelines to prevent supply chain attacks.

Question: Navigate to 10.82.130.51:5003. The code is outdated and imports an old lib/vulnerable_utils.py component. Can you debug it?

I used the command `curl -X POST http://10.82.130.51:5003/api/process -H "Content-Type: application/json" -d '{"data": "debug"}'` and found the flag as **THM{SUPPLY_CH41N_VULN3R4B1L1TY}**

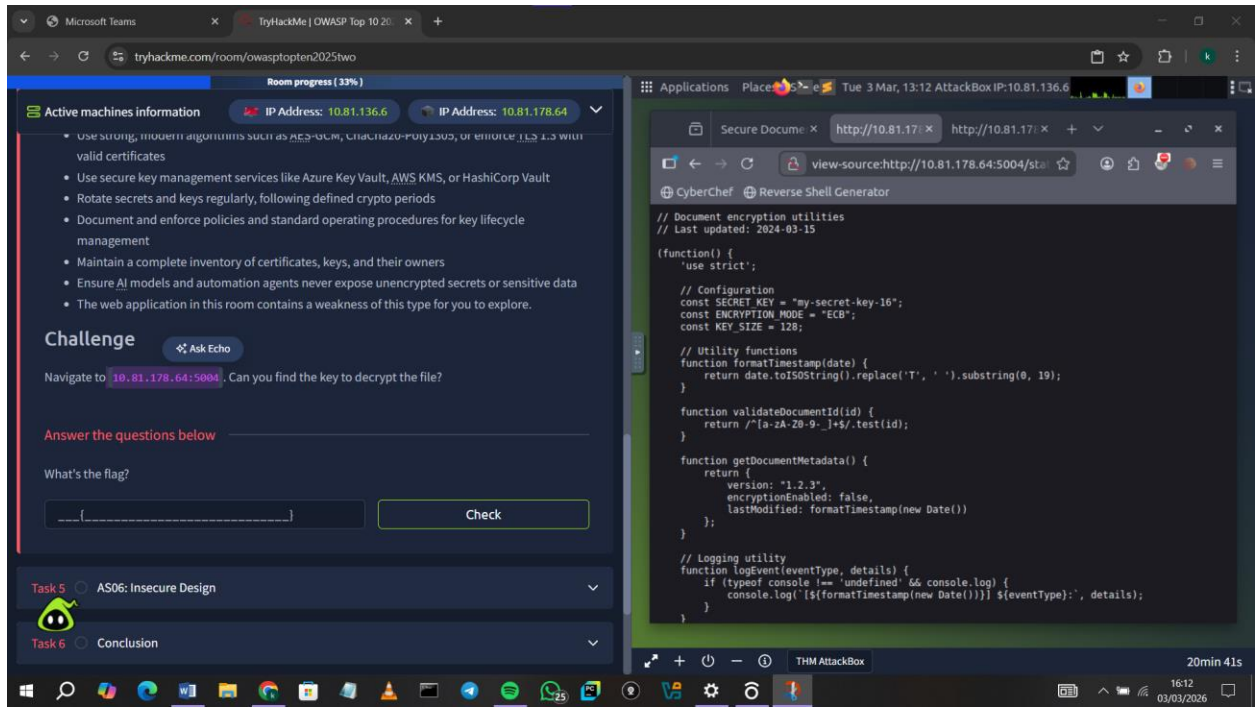


2.3 Cryptographic Failures

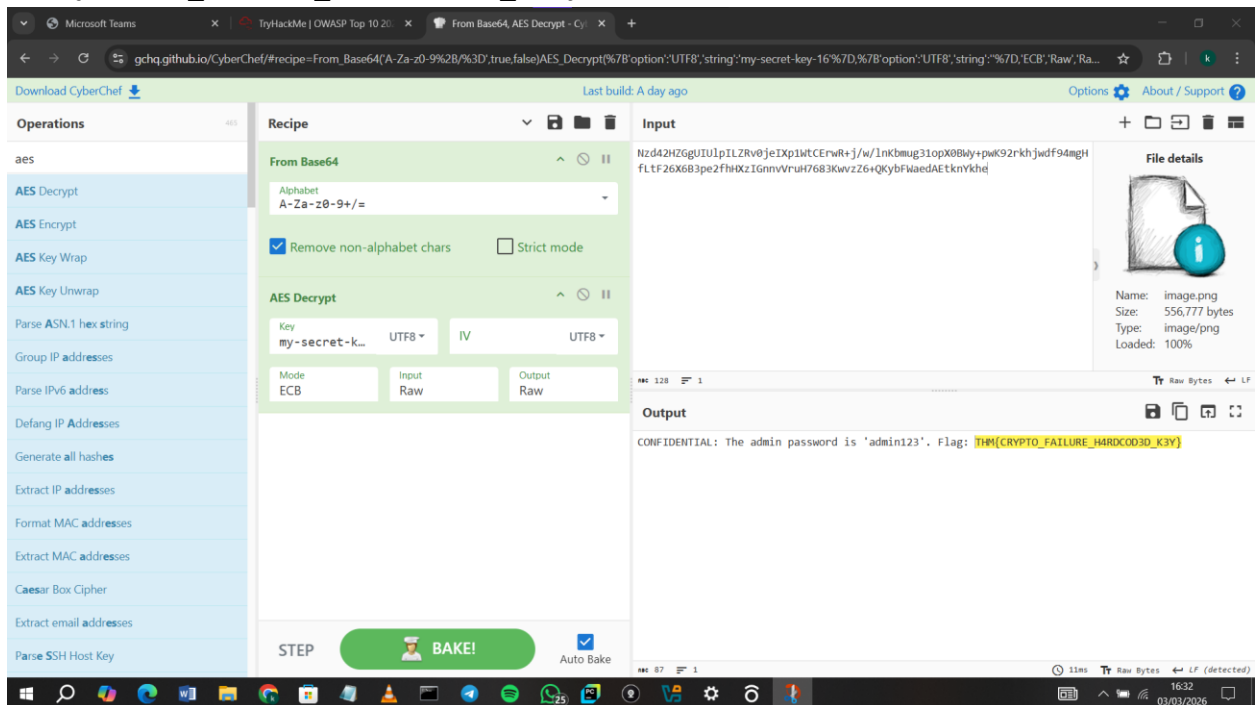
In this section, I learned that Cryptographic Failures occur when encryption is missing, weak, or improperly implemented, exposing sensitive data. I understood how weak algorithms, hard-coded secrets, and poor key management can lead to data breaches and account compromise. I also learned the importance of using strong modern encryption, proper key management, and secure handling of sensitive information.

Question: Navigate to 10.82.130.51:5004. Can you find the key to decrypt the file?

Answer: I viewed the page source then opened the script and found the secret key as **my-secret-key-16** and encryption mode as **ECB**.



I then used **CyberChef** to decrypt and found the flag as
THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}



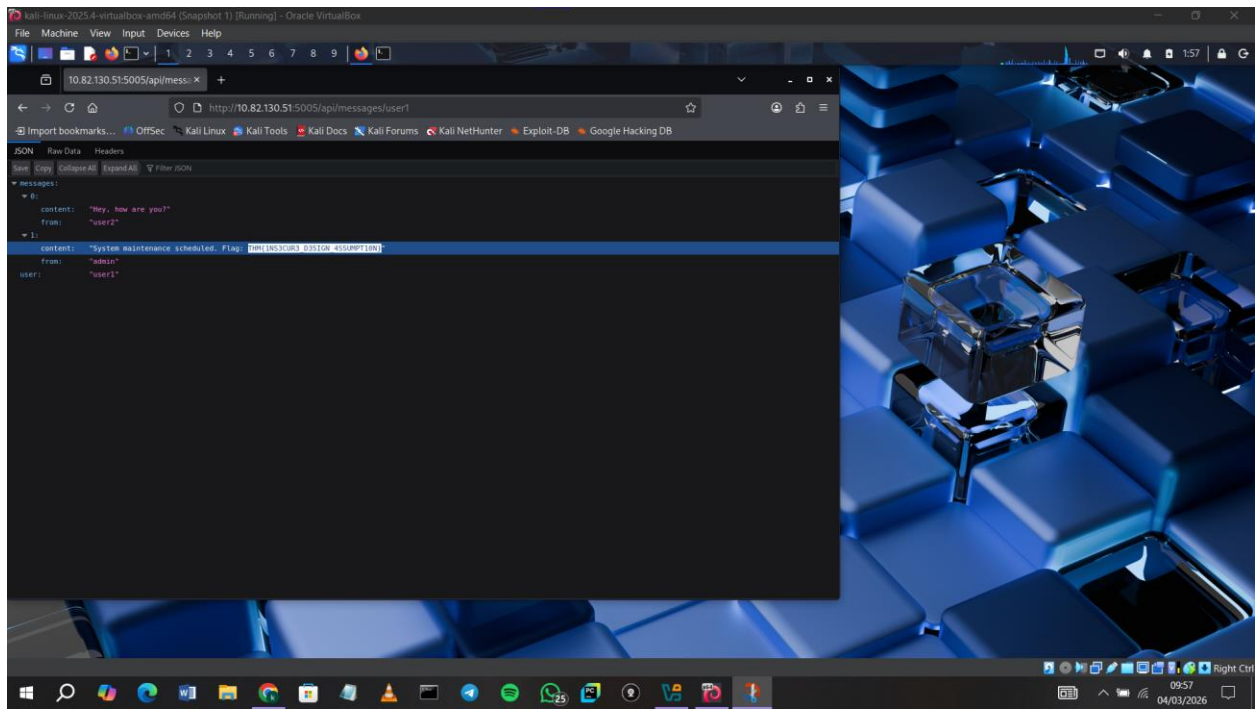
2.4 Insecure Design

In this section, I learned that Insecure Design occurs when security flaws are built into the system's architecture or logic from the beginning. I understood that these

issues cannot simply be patched, as they stem from poor threat modelling, flawed assumptions, or weak business logic. I also learned how AI systems can worsen insecure design when their outputs are blindly trusted or given excessive authority without proper validation and controls.

Question: Navigate to 10.82.130.51:5005. Have they assumed that only mobile devices can access it?

Answer: I opened the address at `/api/messages/user1` and found the flag as **THM{1NS3CUR3_D3SIGN_4SSUMPT10N}**. They **did** make the assumption that only mobile devices can access it.



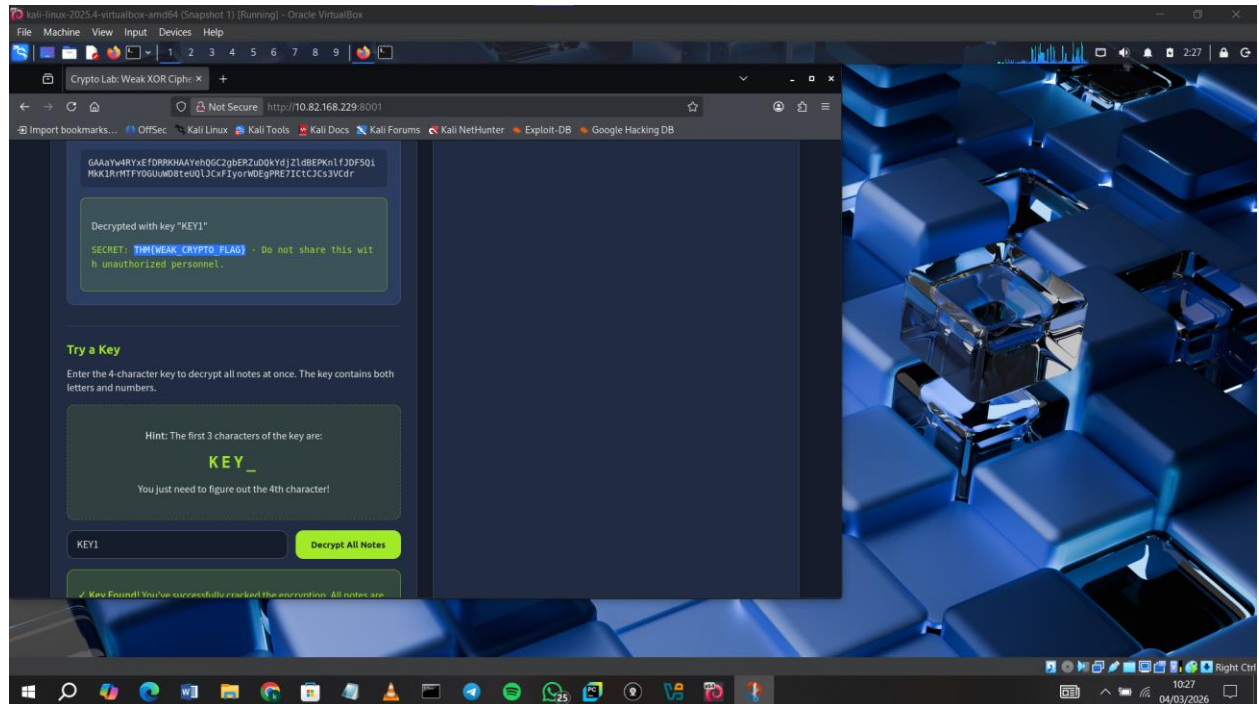
3.0 Insecure Data Handling

3.1 Cryptographic Failures

In this section, I learned that Cryptographic Failures occur when sensitive data is not properly protected due to weak algorithms, poor implementation, or exposed secrets. I understood the risks of storing passwords without secure hashing or creating custom encryption instead of using trusted standards. I also learned the importance of using strong hashing methods like bcrypt or Argon2 and securely managing keys and credentials.

Question: Decrypt the encrypted notes. One of them will contain a flag value. What is it?

Answer: I used **KEY1** to decrypt the notes. The flag was **THM{WEAK_CRYPTO_FLAG}**



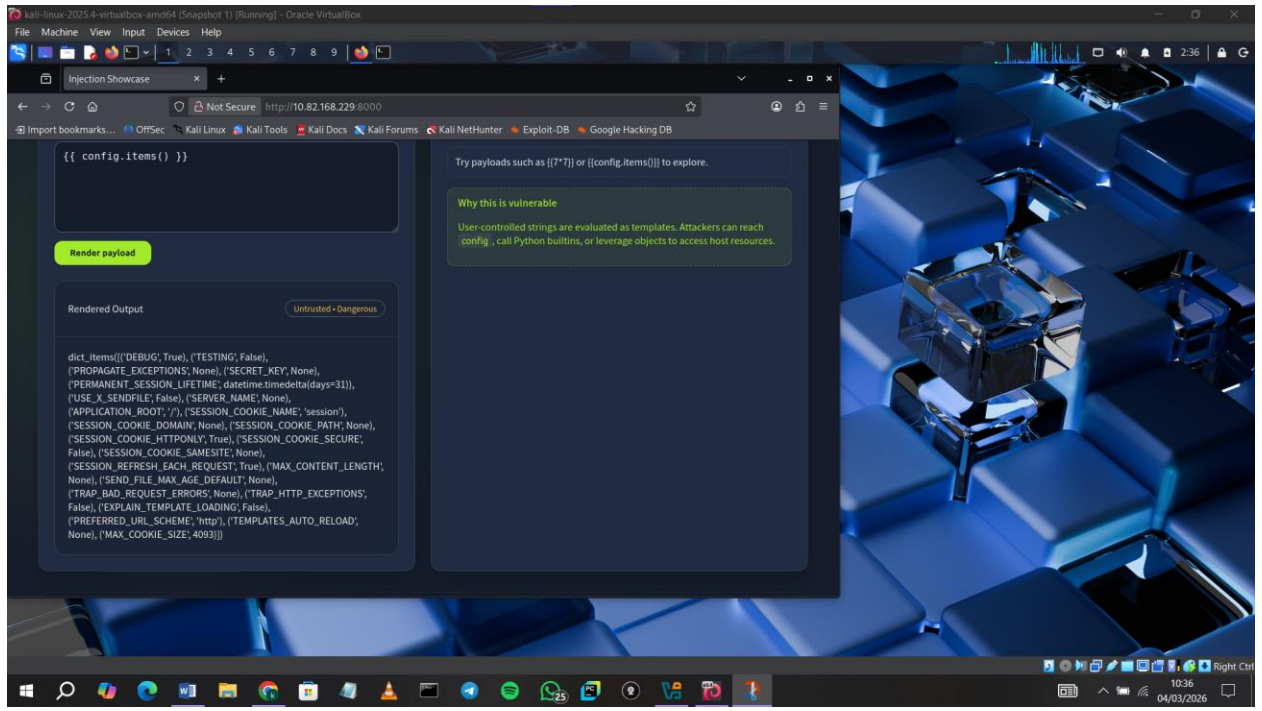
3.2 Injection

In this section, I learned that Injection vulnerabilities occur when applications improperly handle user input and pass it directly into databases, system commands, or templates. I understood how attacks like SQL Injection and Command Injection exploit this flaw to execute malicious queries or commands. I also learned that using parameterised queries, safe APIs, and strict input validation is essential to prevent injection attacks.

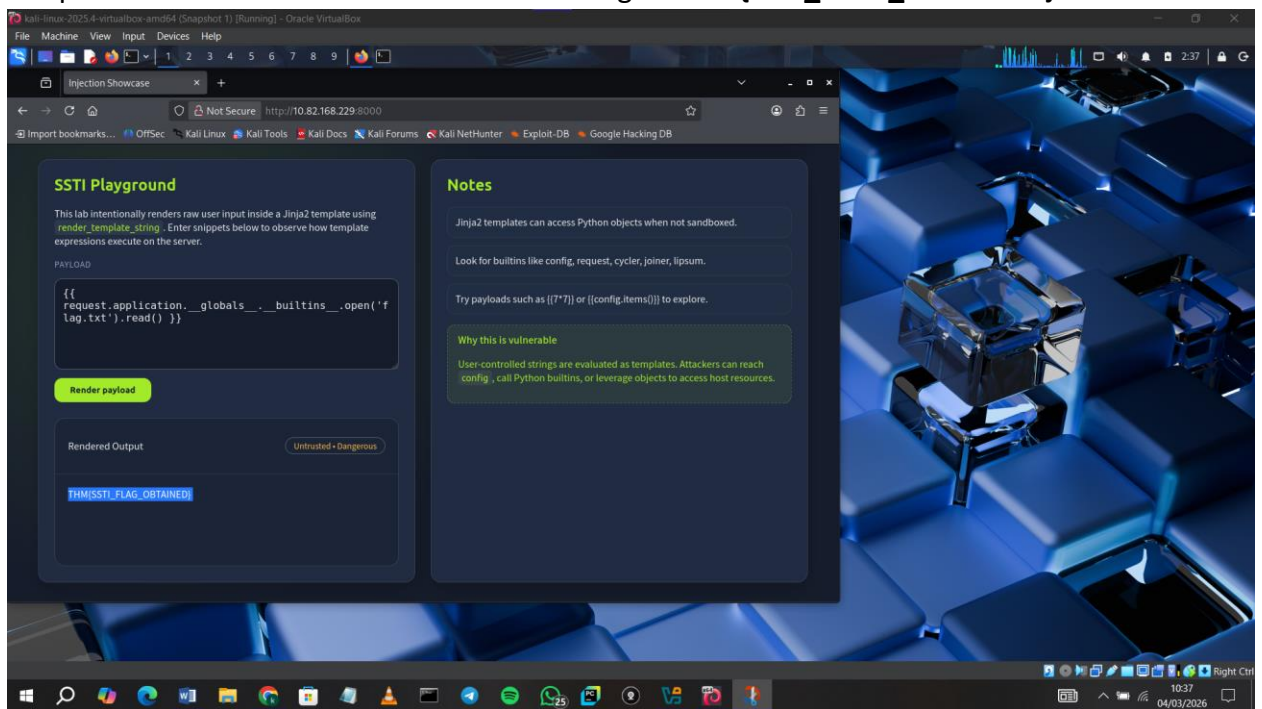
Question: Perform an SSTI attack on the practical. You need to read the contents of flag.txt that is located within the same directory as the web application.

Answer:

I ran `{{ config.items() }}` to discover objects that were exposed to the template.



I then ran `{{ request.application.__globals__.__builtins__.open('flag.txt').read() }}` to open the file on the server and found the flag as **THM{SSTI_FLAG_OBTAINED}**



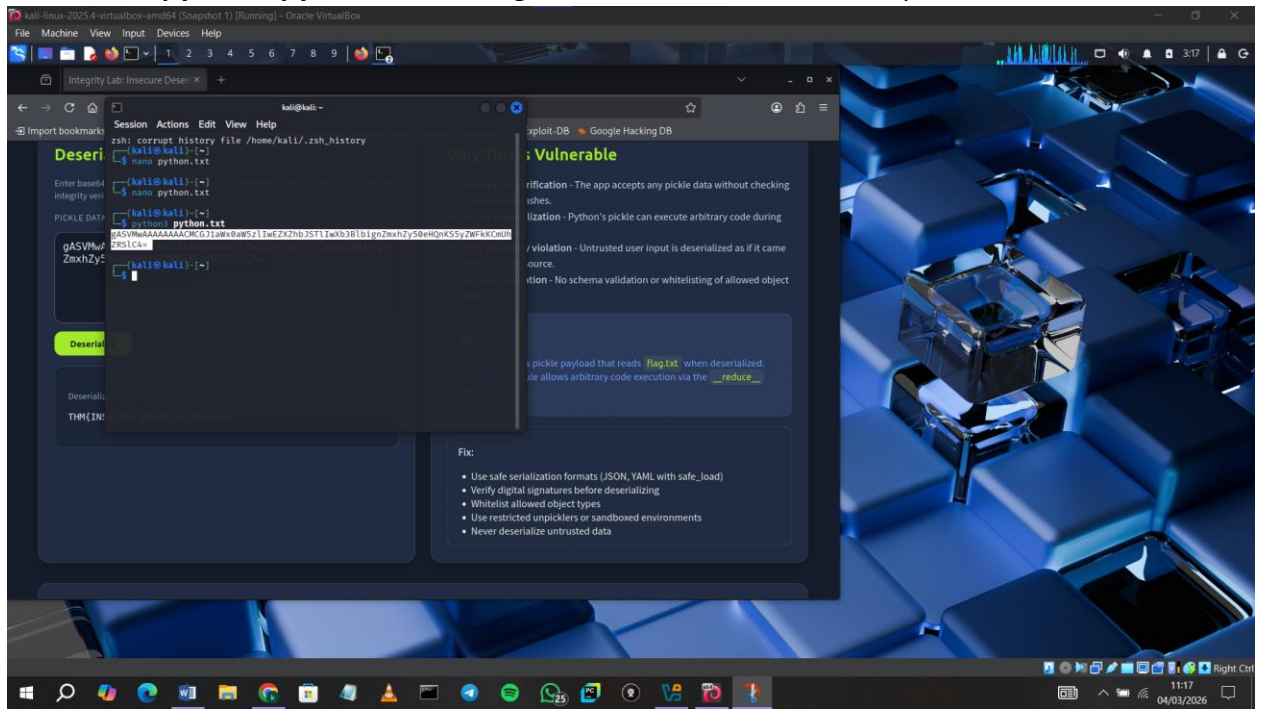
3.3 Software or Data Integrity Failures

In this section, I learned that Software or Data Integrity Failures happen when applications trust code, updates, or critical data without verifying their authenticity

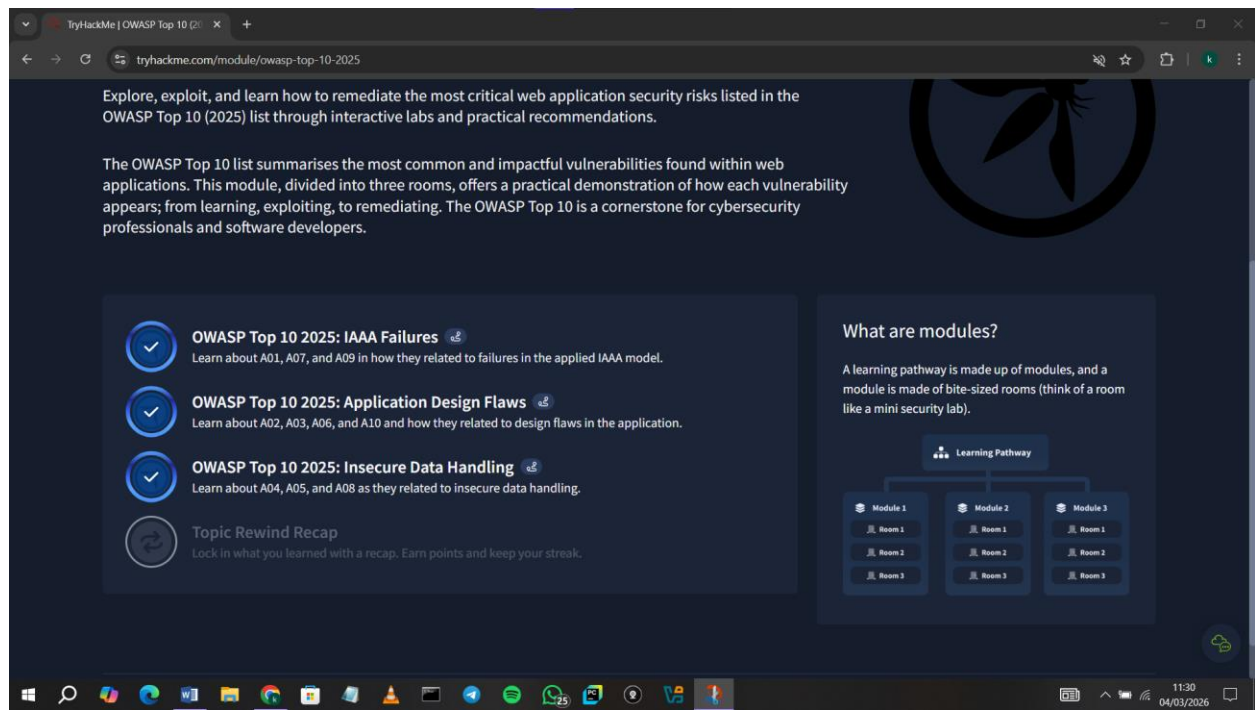
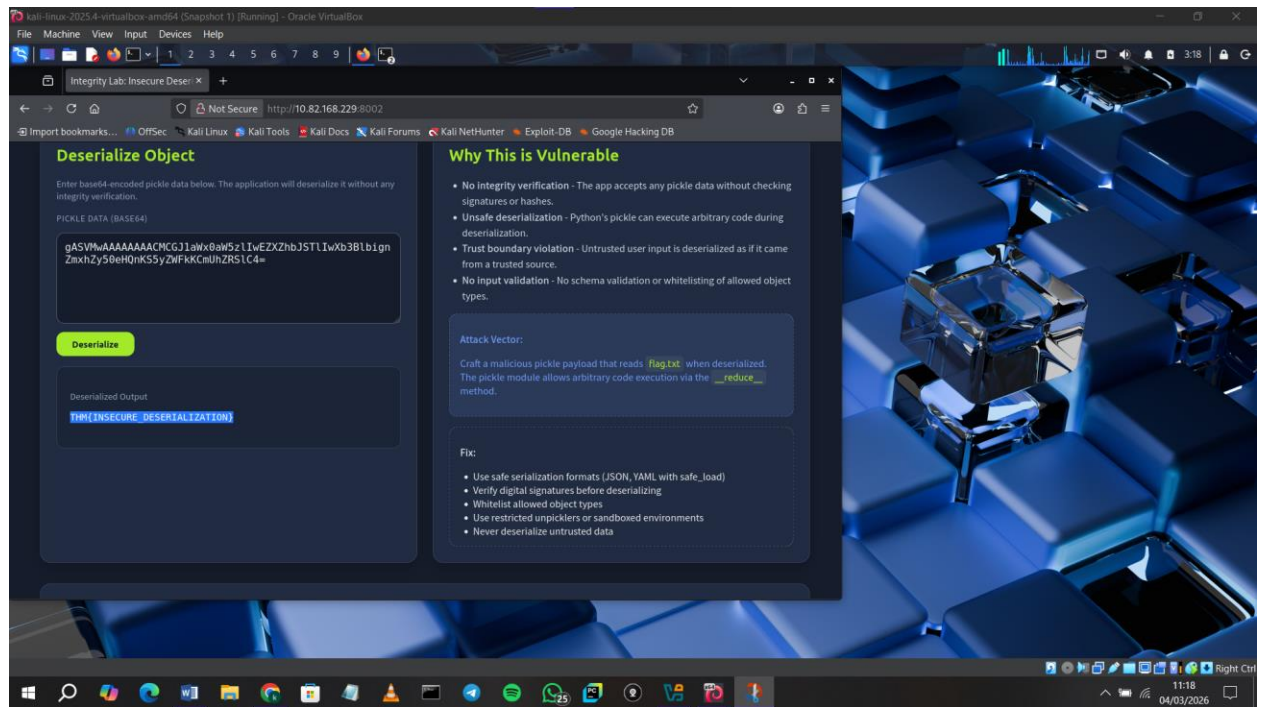
or integrity. I understood that failing to validate sources or check for tampering can allow attackers to inject malicious code or altered data. I also learned the importance of enforcing trust boundaries and using integrity checks, especially within update and CI/CD processes.

Question: Use Python to pickle a malicious, serialised payload that reads the contents of flag.txt and submits it to the application. What are the contents of flag.txt?

Answer: I copied the sample python script and used **nano** to save it in a **python.txt** file and ran **python3 python.txt** which gave me the base64-encoded pickle data.



I deserialized it and obtained the flag as **THM{INSECURE_DESERIALIZATION}**



Conclusion

Overall, this module strengthened my understanding of how real-world web application vulnerabilities occur and how they can be prevented through secure design, proper validation,

strong cryptography, and continuous monitoring. It reinforced the importance of building security into every stage of development rather than treating it as an afterthought.